

Module C Addition: Cross-Platform Word Unification

Component: WordOccurrenceIndex.h | Japanese Speech Transcription & Dictionary Lookup Assistant

1. Objective

The team's Module C brief requires that the same word seen on different platforms or videos map to exactly one hash-table entry, rather than being duplicated per source. Every place the word occurs — identified by a {show, timestamp} pair — must be recorded against that single, unified entry. The deliverable is a working demonstration that saving the same word from two different videos results in both occurrences appearing under one entry.

This was built as a standalone component (WordOccurrenceIndex.h) rather than folded into the existing WordGraph.h, so it can be developed, tested, and reasoned about independently of the kanji/radical relationship graph, and integrated with it (or with Module A/B) later without entangling the two concerns.

2. Design Decisions

2.1 Core data structures

An **Occurrence** struct records a single sighting: the show/video identifier and a timestamp in seconds. A **WordEntry** struct holds the canonical word string plus a **vector<Occurrence>** of every sighting of it. The index itself is an **unordered_map<string, WordEntry>**, keyed by the word — this is what guarantees uniqueness: a given key can only ever map to one WordEntry, so inserting the same word twice can never create a second entry.

2.2 The unification guarantee

recordOccurrence(word, show, timestamp) first checks whether the word already exists in the map. If not, it creates a new WordEntry. Either way, it then appends the new {show, timestamp} pair to that entry's occurrence list. This means repeated calls for the same word — whether from the same show or a different one — always resolve to the same underlying entry and simply grow its occurrence list. No branch in the code path can produce a duplicate entry for an existing word.

2.3 Error-handling convention

Consistent with the convention established in Module B and carried into WordGraph.h, lookups for a word that has never been recorded are a recoverable, expected condition, not an error: lookup() returns nullptr, and occurrenceCount() / showCount() return 0. No exceptions are thrown anywhere in this component.

3. Public API

Method	Returns	Behavior
recordOccurrence(word, show, timestamp)	void	Creates the entry if new; always appends the occurrence.

lookup(word)	const WordEntry*nullptr if the word was never recorded.	
occurrenceCount(word)	size_t	0 if unseen; otherwise total sightings across all shows.
showCount(word)	size_t	0 if unseen; otherwise count of distinct shows it appeared
uniqueWordCount()	size_t	Total distinct words currently tracked.
allWords()	vector<string>	Every distinct tracked word, for reporting/iteration.

4. Complexity Analysis

Operation	Complexity	Notes
recordOccurrence	O(1) amortized	One hash lookup/insert + one vector push_back.
lookup / occurrenceCount / showCount	O(1) amortized	showCount additionally does O(k) work over k occurrences to deduplicate
uniqueWordCount	O(1)	unordered_map::size() is constant time.
allWords	O(n)	n = number of distinct words tracked.

5. C-to-C++ Notes

For a C background: `unordered_map<string, WordEntry>` plays the role a hand-rolled hash table of struct entries would in C (think an array of buckets, each a linked list of {key, struct} nodes) — except the resizing, bucket management, and collision handling are done for you. `WordEntry`'s constructor ("explicit `WordEntry(string wordIn)`") replaces a C-style `init_word_entry()` function: it runs automatically the moment an entry is placed in the map, so there's no risk of forgetting to initialize a field. Returning `nullptr` from `lookup()` is the direct analogue of a C function returning `NULL` for "not found" — same convention, just wrapped in a typed pointer.

6. Deliverable Demonstration

The required demonstration — saving the same word from two different videos and confirming both occurrences appear under one entry — is Test 8 (`testDeliverableDemonstrationScenario`) in the test suite. It records the word `日本語` ("Japanese language") once from `documentary_kyoto.mp4` and once from `gaming_stream_ep5.mp4`, then asserts:

日本語

- Exactly one `WordEntry` exists for the word (`uniqueWordCount() == 1`).
- That single entry's occurrence list has size 2.
- Both source shows (`documentary_kyoto.mp4` and `gaming_stream_ep5.mp4`) are present in that list.

7. Test Summary

`test_word_occurrence_index.cpp` contains 8 self-contained tests covering 36 assertions: new-word creation, cross-video unification (the core guarantee), repeated occurrences within a single show, distinct words never colliding, `nullptr`/zero behavior for unseen words, `allWords()` correctness, empty-index safety, and the

deliverable demonstration scenario itself.

Compiled clean (zero warnings) under: `g++ -std=c++17 -Wall -Wextra -Wshadow -Wconversion -fsanitize=address,undefined`. All 8 tests / 36 assertions pass; AddressSanitizer and UBSan report no issues.

```
$ g++ -std=c++17 -Wall -Wextra -Wshadow -Wconversion -fsanitize=address,undefined \
    -g test_word_occurrence_index.cpp -o test_word_occurrence_index
$ ./test_word_occurrence_index
...
All 8 tests passed (36 assertions).
```

8. Integration Notes for the Team

- No dependency on `NaryTree.h`, `WordGraph.h`, or the stub header — compiles standalone.
- Timestamps are stored as `double` (seconds). If a teammate's transcript module uses a different timestamp representation (e.g. a formatted string), a thin conversion at the call site is all that's needed — no change to this header.
- Whoever wires this into the transcript pipeline (Module B/D) just calls `recordOccurrence(word, showId, timestampSec)` once per recognized word; everything else is handled internally.
- Follows the same `nullptr`-on-failure convention as `WordGraph.h` and Module B, so it composes cleanly with existing error-handling patterns in the codebase.